Repaso rápido: Herencia, Clases Abstractas e Interfaces (Java)

Herencia (`extends`)

- Una **subclase** hereda atributos y métodos de una **superclase**.
- Sintaxis: `class Sub extends Super { ... }`
- Usa `super(...)` para invocar el constructor de la superclase.
- **Sobrescritura**: redefinir (`@Override`) un método de la superclase en la subclase.
- Beneficio: **reusar código** y especializar comportamiento.

Clases Abstractas (`abstract`)

- Una clase puede ser `abstract` si no puede/no debe instanciarse.
- Puede tener:
  - Métodos **abstractos** (sin cuerpo).
  - Métodos **concretos** (con implementación).
- Sintaxis de método abstracto: `public abstract double calcular();`
- Las subclases **concretas** deben implementar todos los métodos abstractos.

> **Uso típico:** definir una base común (atributos + métodos compartidos) y obligar a implementar detalles en las subclases.

Interfaces (`implements`)

- Son “contratos”: definen métodos que la clase debe implementar.
- Una clase puede implementar **varias interfaces**:
  `class X implements I1, I2 { ... }`
- Métodos en interfaces (Java 8+): pueden ser `default` (con cuerpo) o abstractos.
- Las interfaces no tienen estado (antes de Java 8, no tienen atributos “normales”).

> **Tip:** usar interfaces para comportamientos “transversales” (p.ej., `Encendible`, `ConectableWiFi`).

Tip rápido de polimorfismo

- Declara variables del tipo de la **superclase** o **interfaz**.
- Al ejecutar `obj.metodo()`, se usa la implementación real de la clase concreta.
- Ejemplo:

```
FiguraGeometrica f = new Rectangulo(...);
f.calcularArea(); // usa Rectangulo.calcularArea()
```

Errores comunes a evitar

- Declarar una clase abstracta y luego intentar instanciarla (`new ClaseAbstracta()`).
- Olvidar `@Override` al sobrescribir y, por accidente, no coincidir firma.
- Crear una clase que implementa una interfaz y no definir todos sus métodos (a menos que la clase sea abstracta).
- Confundir `extends` (herencia) con `implements` (interfaces).

Tips para examen

1. **Identificar**: ¿esto es “comportamiento” (interface) o “relación es-un” (herencia)?
2. Si piden “reusar código + obligar a implementar en hijos”, usa **clase abstracta**.
3. Si piden “varias capacidades independientes”, usa **interfaces**.
4. Dibujar rápido la jerarquía (clase base → subclases) antes de escribir el código.